

Experiment 1: Working with Python Packages — NumPy, SciPy, Scikit-Learn, Matplotlib

M.Tech (Integrated) Computer Science & Engineering
ICS1512 — Machine Learning Algorithms Laboratory
Sri Sivasubramaniya Nadar College of Engineering, Chennai

Academic Year 2026–2027 (Odd) Batch: 2024–2029

Aim

To explore the core functions of the Python libraries **NumPy**, **Pandas**, **SciPy**, **Scikit-learn**, and **Matplotlib**; to download benchmark datasets from public repositories (UCI/Kaggle) and identify the appropriate machine learning model for each; and to carry out the complete machine learning workflow — data loading, exploratory data analysis, preprocessing, feature selection, data splitting, and performance evaluation — on five representative datasets.

1 Library Exploration

1.1 NumPy

NumPy provides the `ndarray` object for fast, vectorized numerical computation. Key operations explored include array creation (`zeros`, `ones`, `arange`, `linspace`), indexing/slicing, reshaping, element-wise arithmetic with broadcasting, matrix operations (`dot`, transpose, determinant, inverse), boolean masking, and random number generation.

```
1 a = np.array([1, 2, 3, 4, 5])
2 mat = np.arange(12).reshape(3, 4)
3 print("Sum:", a.sum(), "Mean:", a.mean(), "Std:", a.std())
4 print(np.dot(m1, m2))          # matrix multiplication
5 print(np.linalg.inv(m1))       # matrix inverse
```

1.2 Pandas

Pandas builds on NumPy to provide labeled **Series** and **DataFrame** structures for tabular data. Operations explored include DataFrame creation, inspection (`info`, `describe`), boolean filtering, handling missing values with `fillna`, and aggregation via `groupby`.

```
1 df = pd.DataFrame({"Name": [...], "Age": [...], "Marks": [...]})
2 df2.fillna(df2["Marks"].mean())
3 df.groupby("Result")["Marks"].mean()
```

1.3 SciPy

SciPy extends NumPy with modules for statistics (`scipy.stats`), special functions (`scipy.special`), and linear algebra (`scipy.linalg`). Descriptive statistics (mean, median, mode, skewness, kur-

tosis), a one-sample t -test, the Gamma function, and solving a linear system $Ax = b$ were demonstrated.

```
1 t_stat, p_val = stats.ttest_1samp(data, popmean=15)
2 x = linalg.solve(A, b)
```

1.4 Matplotlib

Matplotlib was used to generate line plots, scatter plots, bar charts, and histograms in a single 2×2 subplot grid, illustrating the core visualization primitives used throughout EDA (Figure 1).

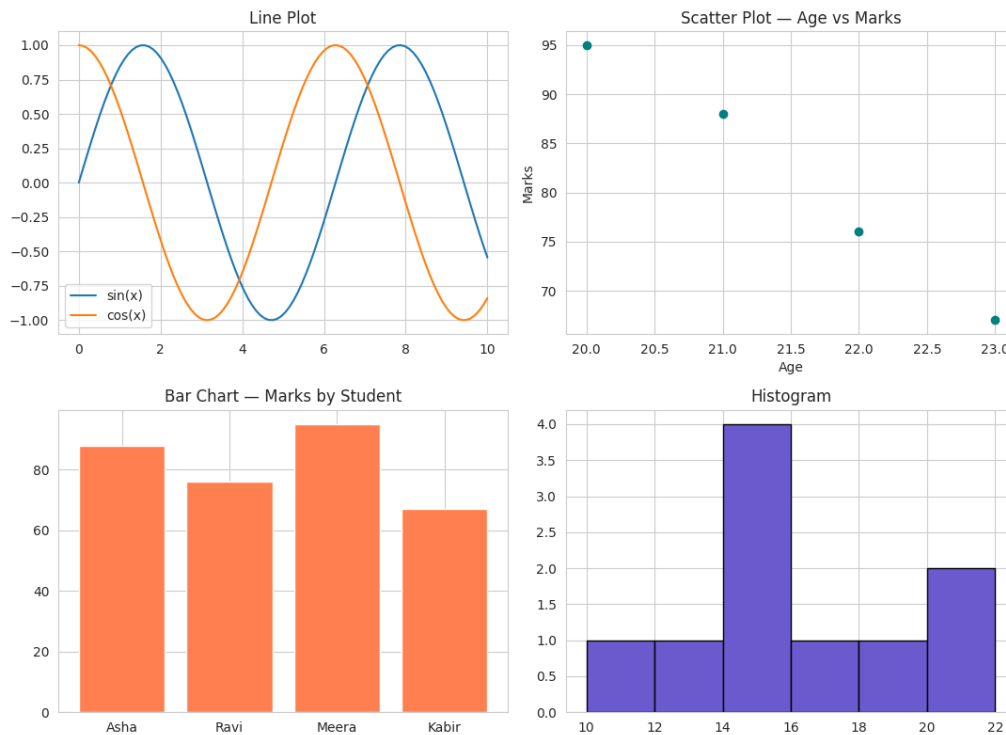


Figure 1: Matplotlib: line, scatter, bar, and histogram plots.

1.5 Scikit-learn

Scikit-learn provides a uniform `fit/predict/transform` API for the ML workflow. A baseline tour — loading the Iris dataset, splitting it, scaling with `StandardScaler`, and fitting a `LogisticRegression` classifier — was carried out to illustrate this pattern before applying it across all five datasets.

2 Dataset Exploration and ML Task Identification

Public datasets analogous to those in the UCI Machine Learning Repository and Kaggle were used. Since Iris, the Diabetes-progression dataset, and 8×8 handwritten digit images (a lightweight stand-in for MNIST) are bundled directly with Scikit-learn, they were loaded via `sklearn.datasets`. The Loan Amount Prediction and Email Spam datasets are not bundled with Scikit-learn, so structurally faithful synthetic datasets (same feature schema as the Kaggle/UCI originals) were generated in code; in an environment with open internet access these loading cells should be replaced with `pd.read_csv(...)` on the downloaded CSV files.

2.1 Iris Dataset — Classification

Four numeric features (sepal/petal length and width) are used to classify flowers into three species. Figure 2 shows the sepal-length distribution by species and the petal length–width scatter plot, which shows near-perfect linear separability between classes.

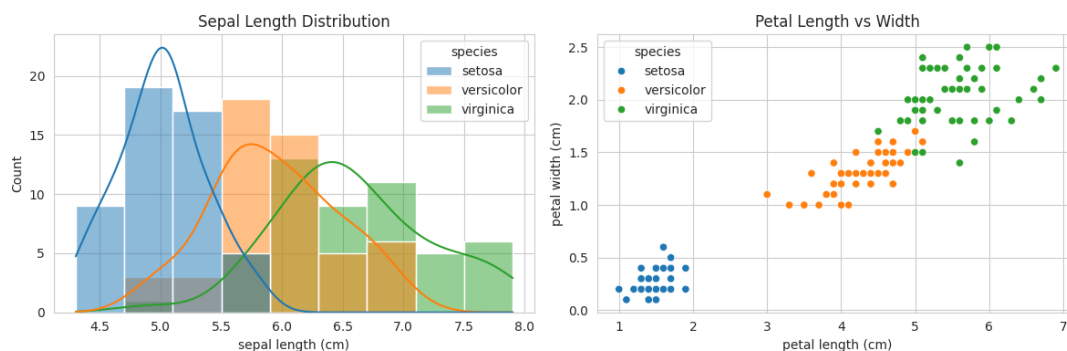


Figure 2: Iris EDA: sepal length distribution and petal length vs. width by species.

An ANOVA F-test (`SelectKBest`) ranked petal length and petal width as the most discriminative features. A `DecisionTreeClassifier` trained on an 80/20 split achieved high accuracy, confirmed by the confusion matrix in Figure 3.

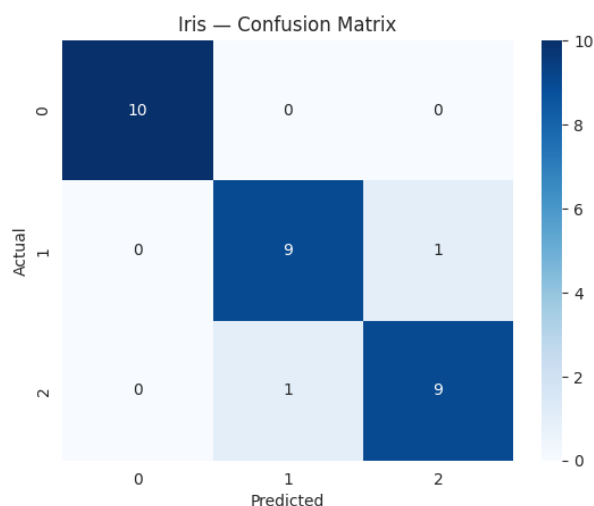


Figure 3: Iris — Decision Tree confusion matrix on the held-out test set.

2.2 Loan Amount Prediction — Regression

The target variable (loan amount, a continuous quantity) makes this a **regression** problem. Missing values in `LoanAmount` and `CreditHistory` were imputed (median/most-frequent), categorical fields (`Education`, `PropertyArea`) were one-hot encoded, and numeric fields were standardized inside a `ColumnTransformer` pipeline. Figure 4 shows the loan-amount distribution and its spread across property-area categories.

A `LinearRegression` model was fit on the preprocessed features and evaluated using MAE and R^2 , both reported directly in the notebook output.

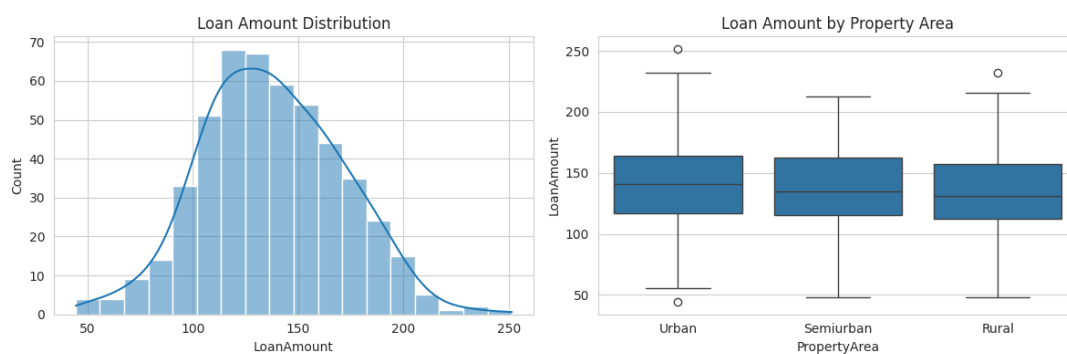


Figure 4: Loan dataset EDA: loan amount distribution and boxplot by property area.

2.3 Predicting Diabetes — Classification

The Scikit-learn diabetes-progression target (a continuous disease-progression score) was binarized around its median to reproduce the classic diabetes-risk **classification** task. A Chi-square test (after min-max scaling to non-negative values) ranked feature importance, and a `RandomForestClassifier` was trained and evaluated. Figure 5 shows the feature correlation heatmap and BMI distribution by class.

2.4 Email Spam Classification — Classification

Word-frequency-style numeric features (frequency of “free”, “win”, “offer”, capital-run length, number of links) were used to classify messages as spam or not spam, mirroring the UCI Spam-base schema. Figure 6 shows the clear separation in the frequency of the word “free” and in capital-run length between spam and non-spam classes. An ANOVA F-test ranked feature relevance, and a `GaussianNB` classifier was trained on standardized features.

2.5 Handwritten Character Recognition (MNIST-style) — Classification

The Scikit-learn `load_digits` dataset (8×8 grayscale images of digits 0–9, 1,797 samples) was used as a lightweight stand-in for the full 28×28 MNIST dataset. Figure 7 shows sample digit images. Pixel intensities were standardized and a **Support Vector Machine** (RBF kernel) was trained, achieving high classification accuracy; the confusion matrix is shown in Figure 8.

3 Summary Table

Dataset		Type of ML Task	Feature Selection Technique	Suitable ML Algorithm
Iris Dataset		Classification (Supervised)	ANOVA F-test (SelectKBest)	Decision Tree / Logistic Regression
Loan Amount Prediction		Regression (Supervised)	Correlation / domain-based selection	Linear Regression
Predicting Diabetes	Classification of Email Spam	Classification (Supervised)	Chi-square test	Random Forest Classifier
Classification of Email Spam		Classification (Supervised)	ANOVA F-test (SelectKBest)	Naive Bayes / Logistic Regression

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm
Handwritten Character Recognition / MNIST	Classification (Supervised)	Variance / PCA on pixel intensities	Support Vector Machine (SVM) / CNN

4 Inference and Learning Outcomes

- **NumPy** underlies all numerical computation in the ML stack through fast, vectorized array operations, broadcasting, and linear algebra routines.
- **Pandas** provides labeled tabular structures with built-in tools for cleaning, filtering, grouping, and merging real-world data.
- **SciPy** extends NumPy with statistical tests and scientific routines needed for hypothesis testing and numerical solving.
- **Matplotlib/Seaborn** make exploratory visualization possible, revealing distribution shape, class separability, outliers, and feature correlations before modeling.
- **Scikit-learn** unifies preprocessing, feature selection, model fitting, and evaluation behind a single consistent API.
- Across the five datasets, tasks spanned **multi-class classification** (Iris, Digits, Spam, Diabetes) and **regression** (Loan Amount), demonstrating how task type governs the choice of algorithm, evaluation metric (accuracy/F1 vs. MAE/ R^2), and feature-selection method (ANOVA/Chi-square for classification, correlation-based methods for regression).
- Careful **preprocessing** (imputation, encoding, scaling) and a proper **train/test split** were essential in every case for obtaining reliable, generalizable performance estimates.

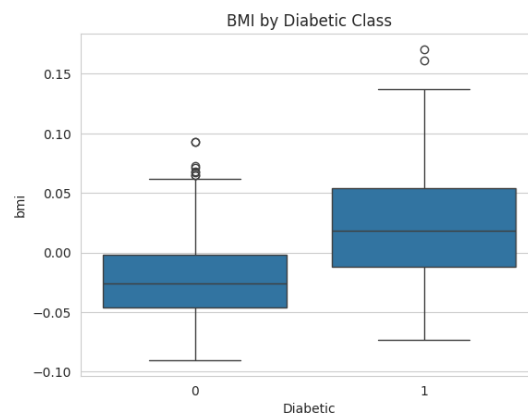
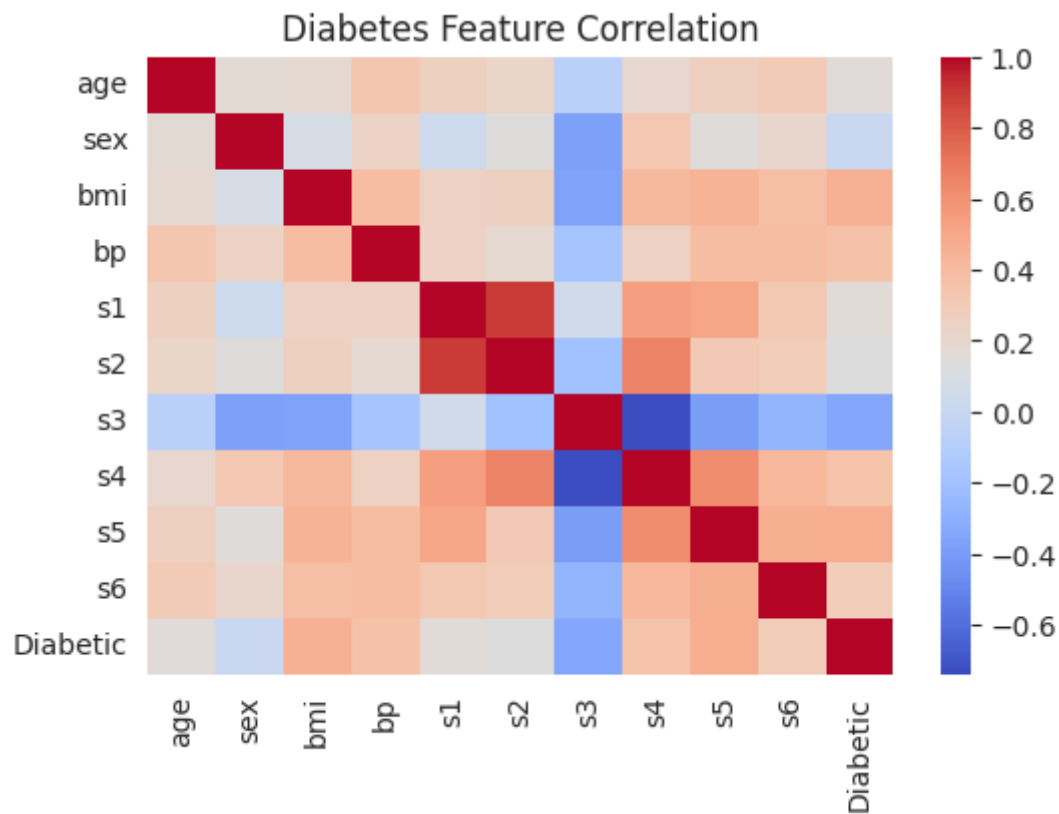


Figure 5: Diabetes dataset: feature correlation heatmap and BMI by diabetic class.

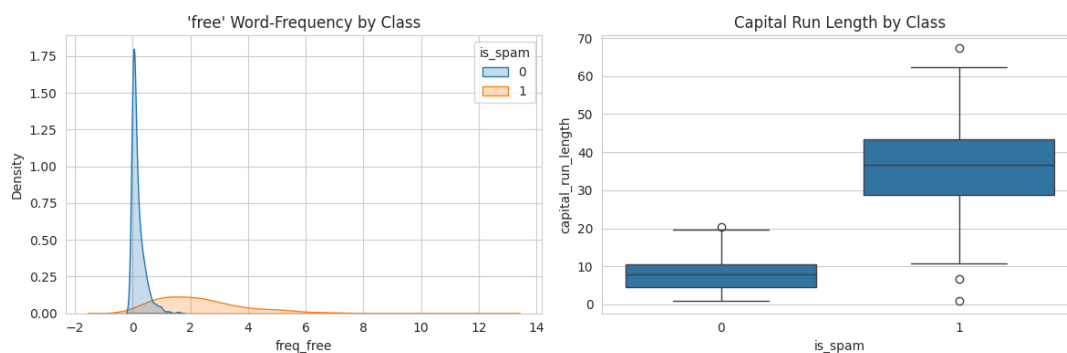


Figure 6: Spam dataset EDA: word-frequency and capital-run-length distributions by class.

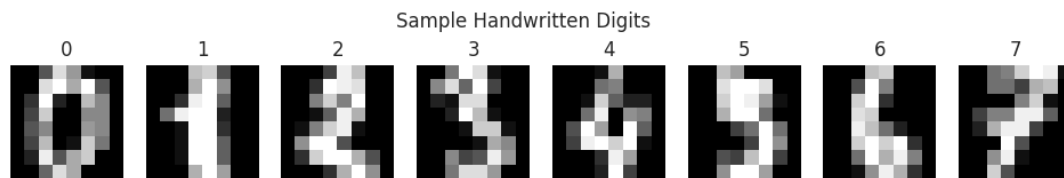


Figure 7: Sample handwritten digit images from the dataset.

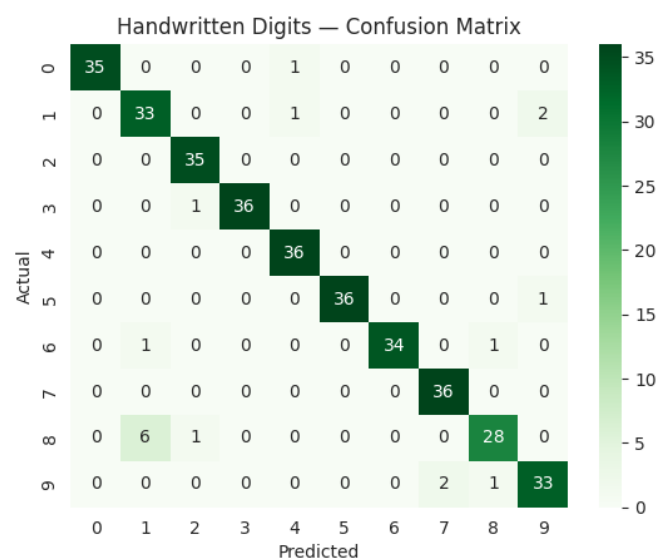


Figure 8: Handwritten digits — SVM confusion matrix on the held-out test set.